

Attention Is All your Need
- Ashish Vaswani 2017
etc

machine translation task

e.g. English sentence → German sentence

= prompt

= question

response

answer

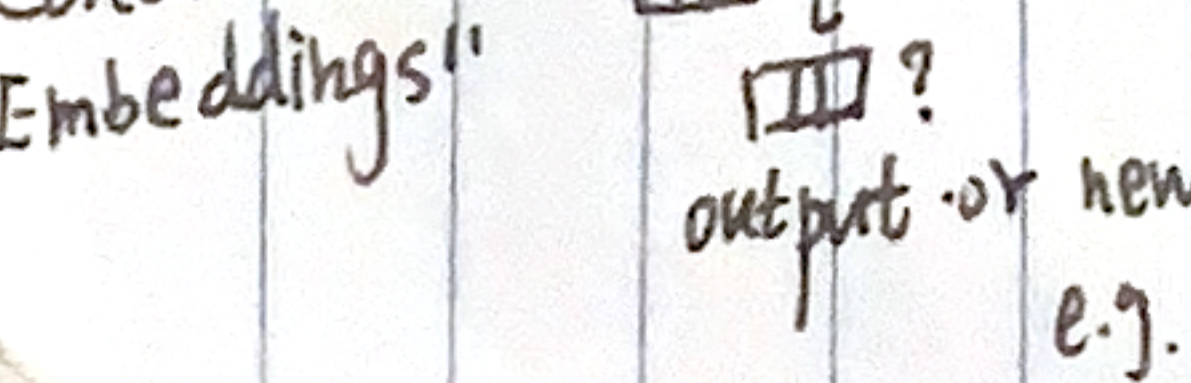
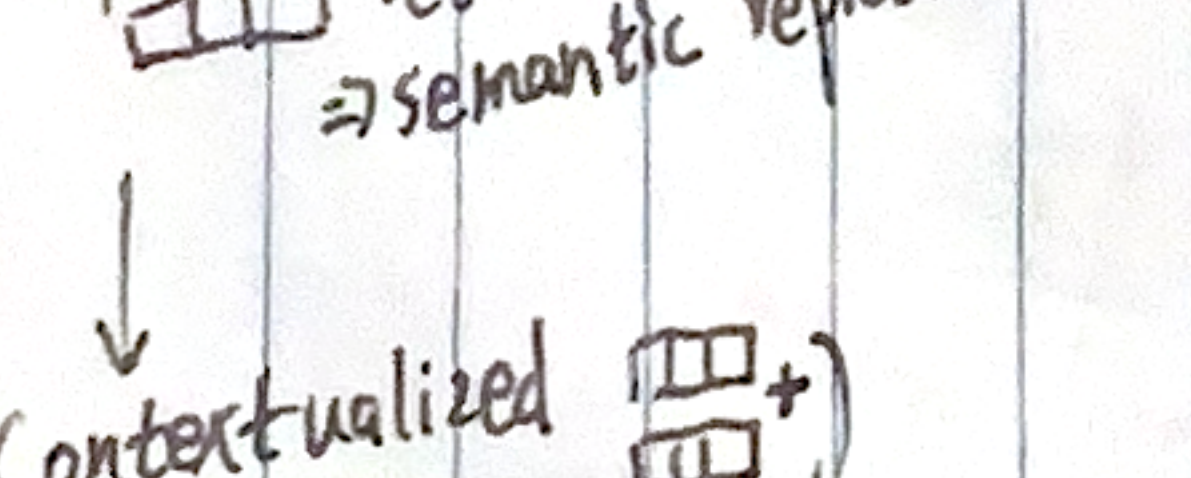
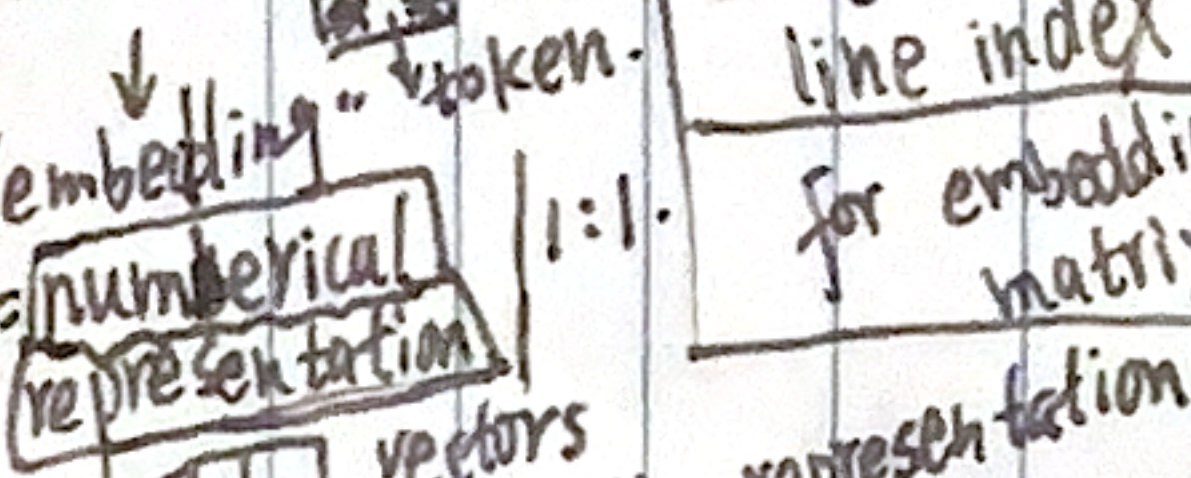
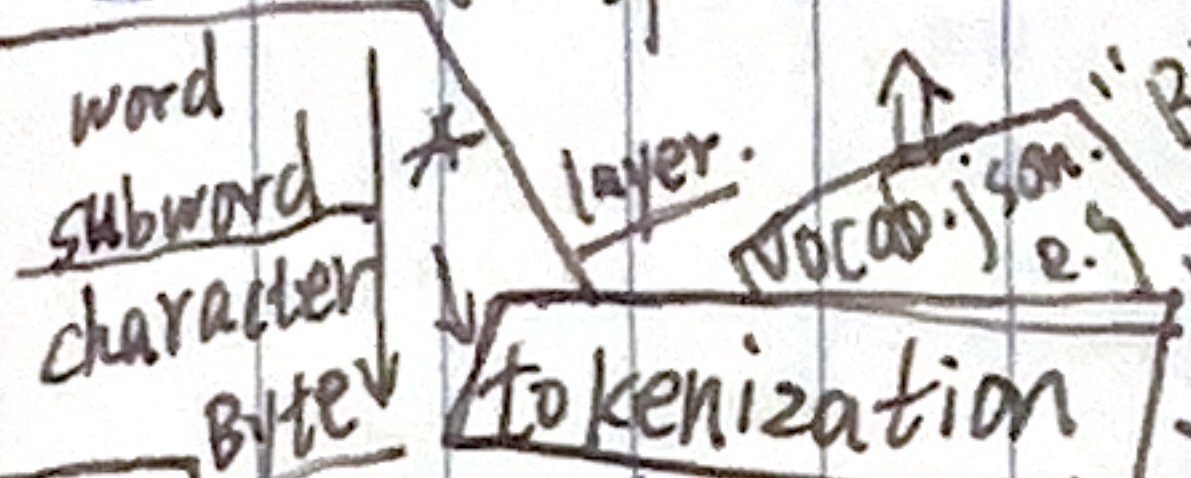
generative text
Decoder

Basis "BERT" model

LLMs from OpenAI

= Bidirectional Encoder Representations from transformers

Anthropic, Cohere and Meta.



⇒ history of Modern LLMs

class {
□ non-transformer models = unstructure data
□ encoder-only = language → numerical component.
+ decoder-only = generative
□ encoder-decoder

□ -2000 Bag of Words

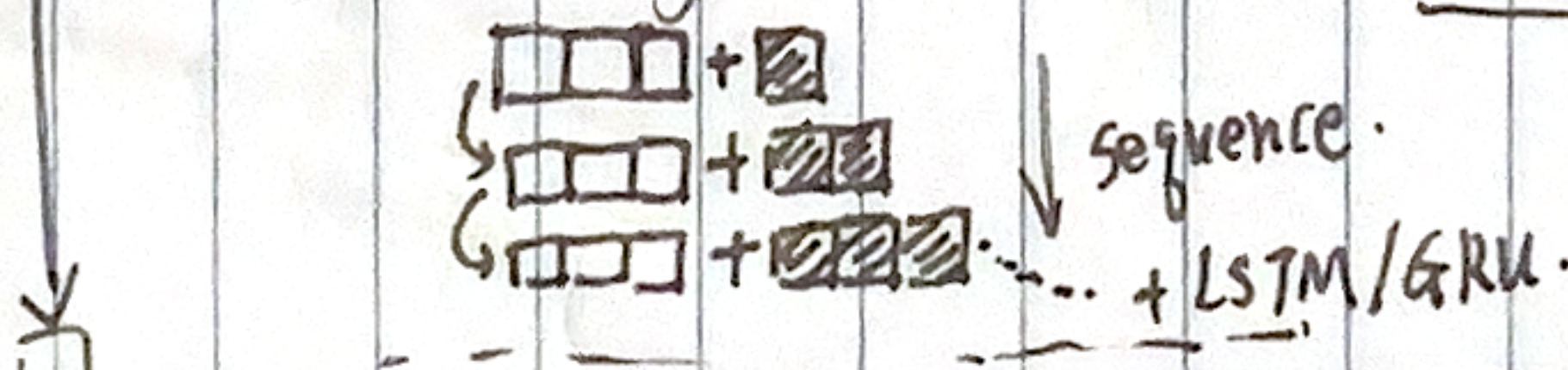
+ semantic
= meaning of words in vector embeddings

□ -2013 Word2Vec
(training data e.g Wikipedia)

platform

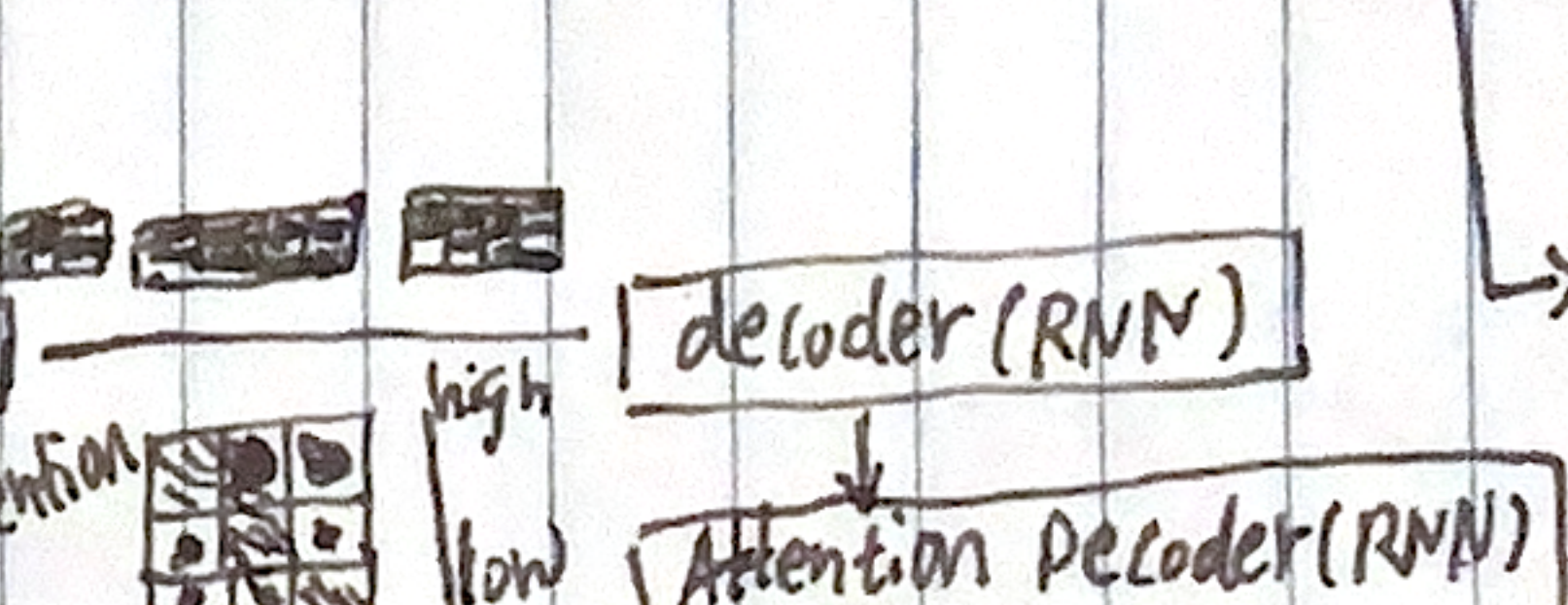
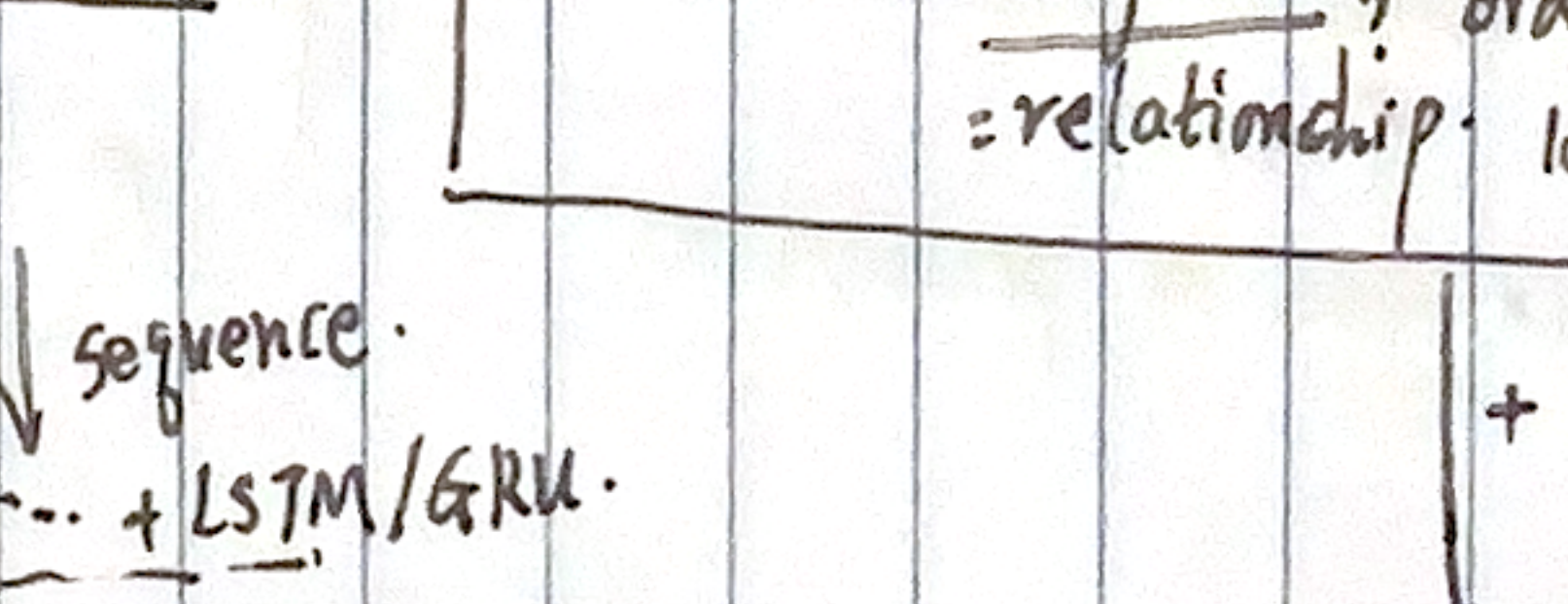
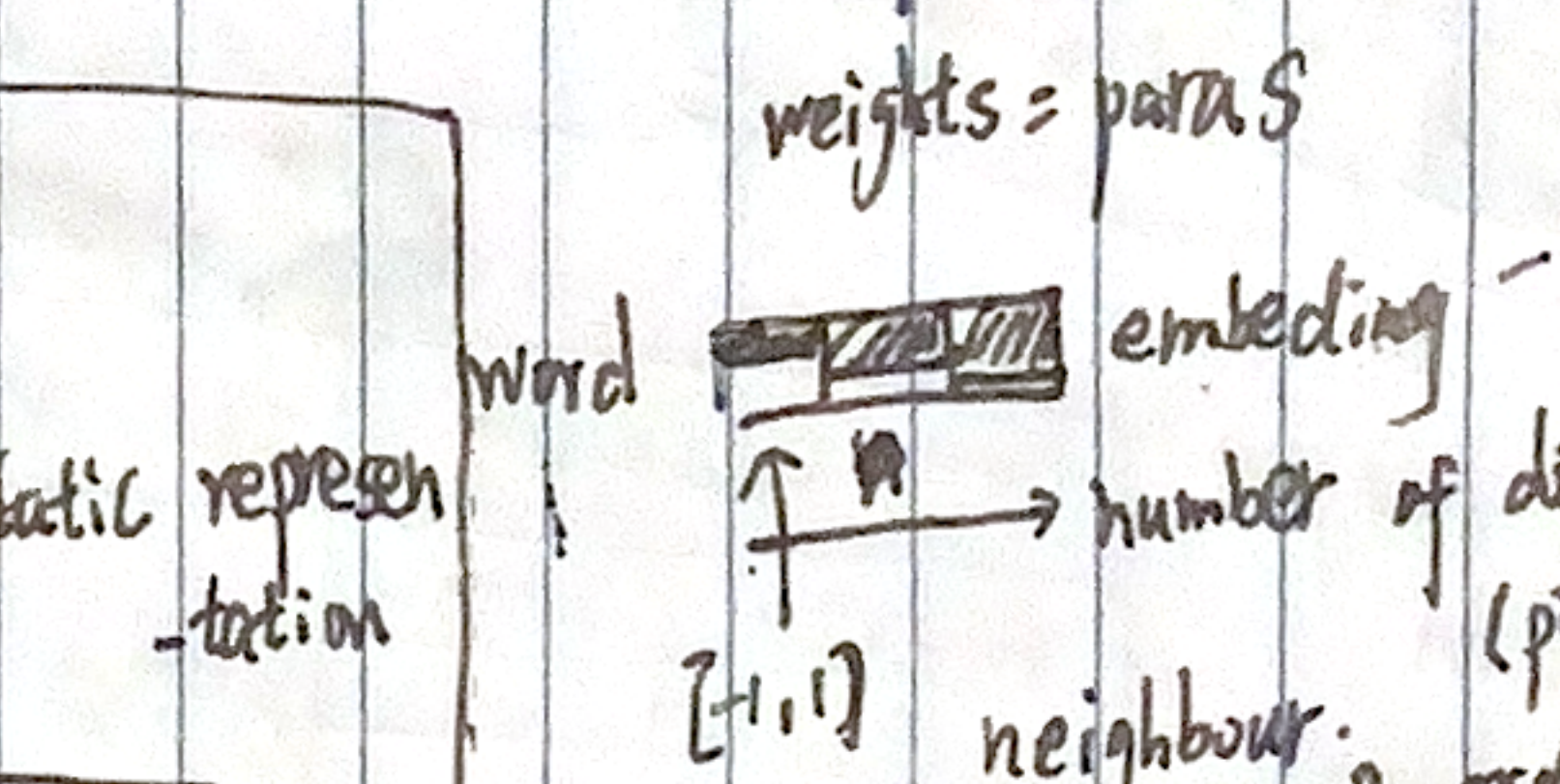
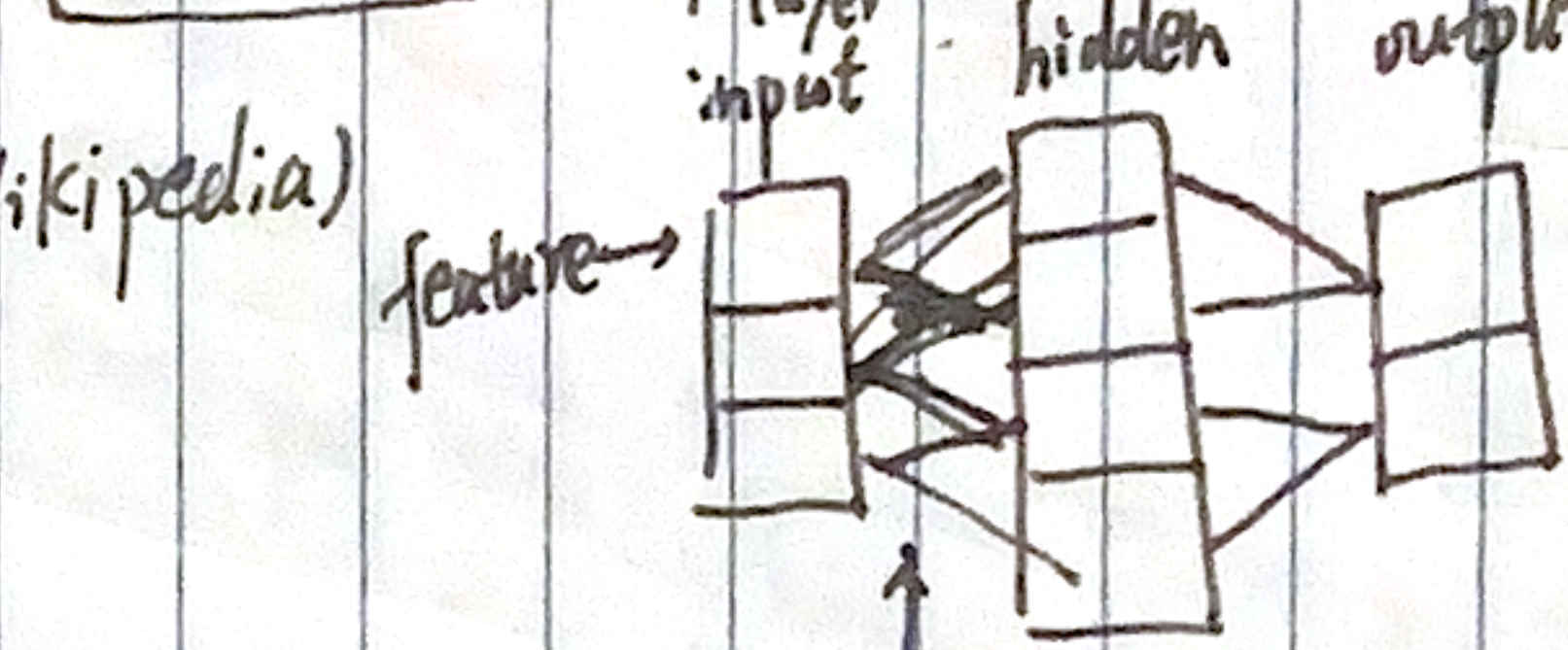
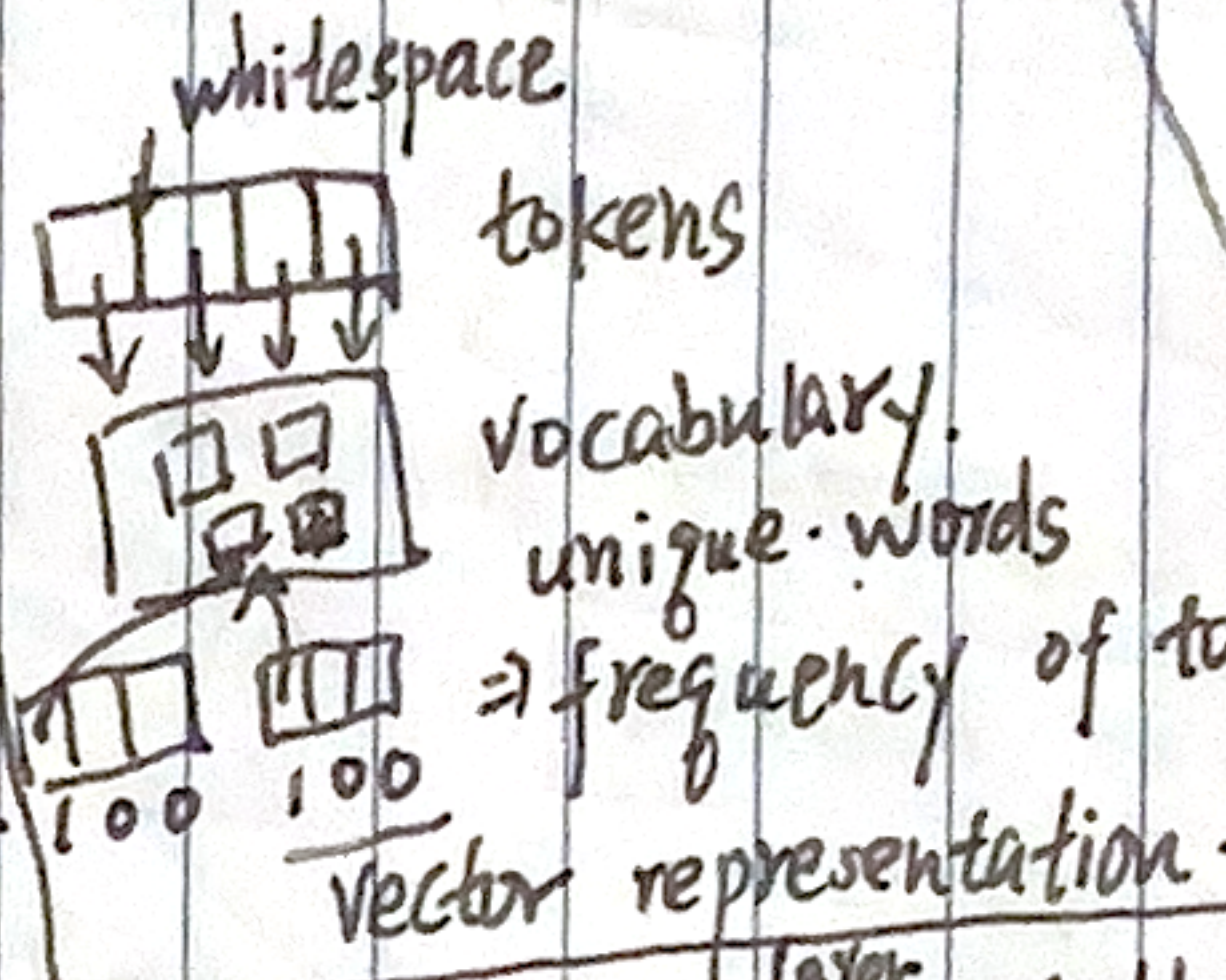
+ contextualized information
rather instead of static representation
= attention.

Autoregressive



"RNN"

Recurrent Neural Networks
⇒ entire sequences



Flan-T5
↑
switch
↑
T5

ROBERTA

Distil BERT

Generative pre-training "GPT"

Transformer

2018

transformer pretraining + fine-tune downstream

BERT [CLS] [SEP] and separator flag

masked classification for token prediction

predic e.g emotion prediction

representation models

document sentence word token subword

max context length

Generative models

attention

transformer decoder

masked self-atten

+ parallel

transformer encoder

self-attention

FFN

feed forward Neural network

encoder attention

decoder(RNN)

Attention Decoder(RNN)

chat GPT

175B

1.5B

117M

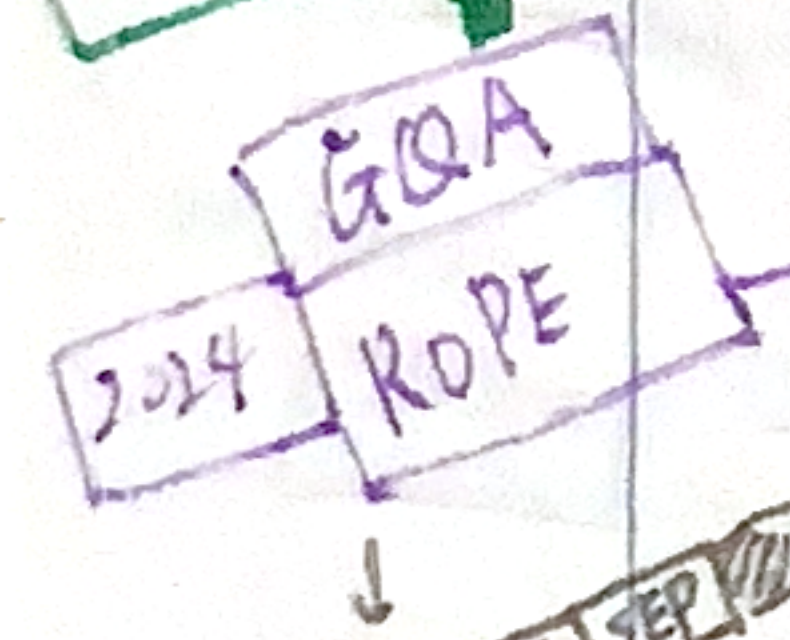
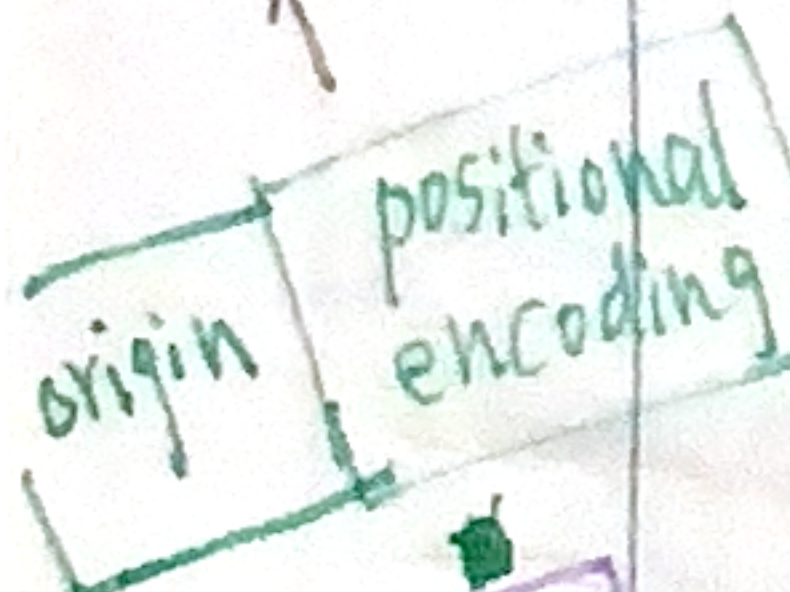
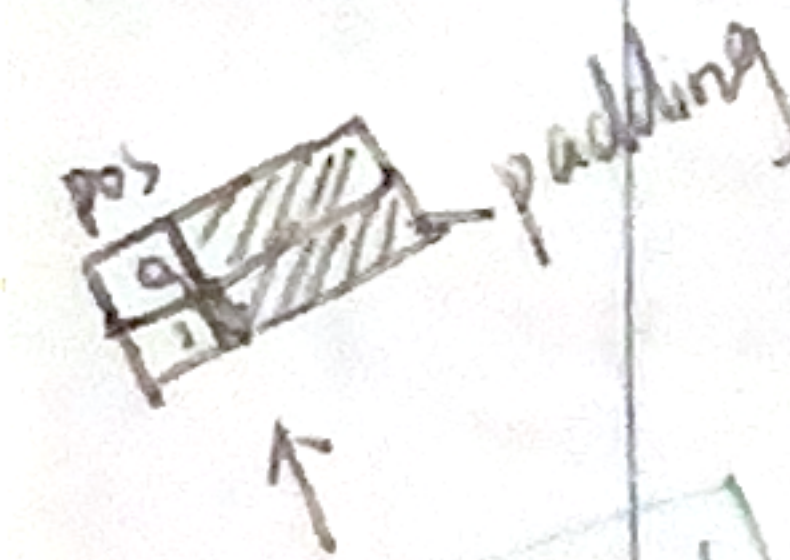
num of paras

intuition about transformers workflow

major parts

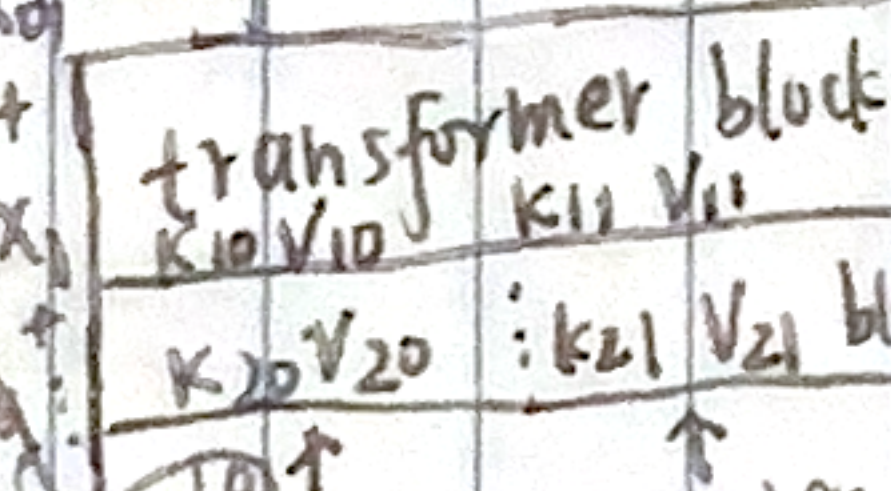
Tokenizer

Find the tokenID from vocab.json



Calculation.
pos: m
index: i
 $\theta = m / (\text{lower_id})$
for (a, b).
 $a' = a * \cos(\theta) - b * \sin(\theta)$
 $b' = a * \sin(\theta) + b * \cos(\theta)$

stack of transformer blocks

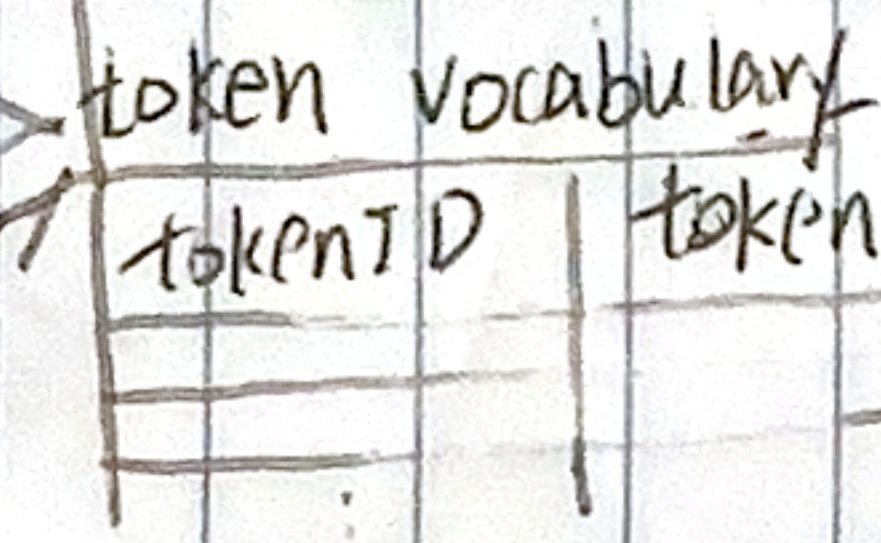


LM Head

output

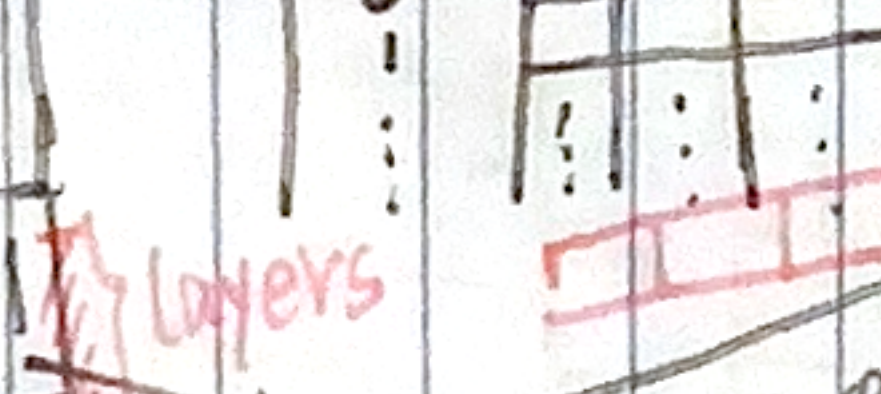
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



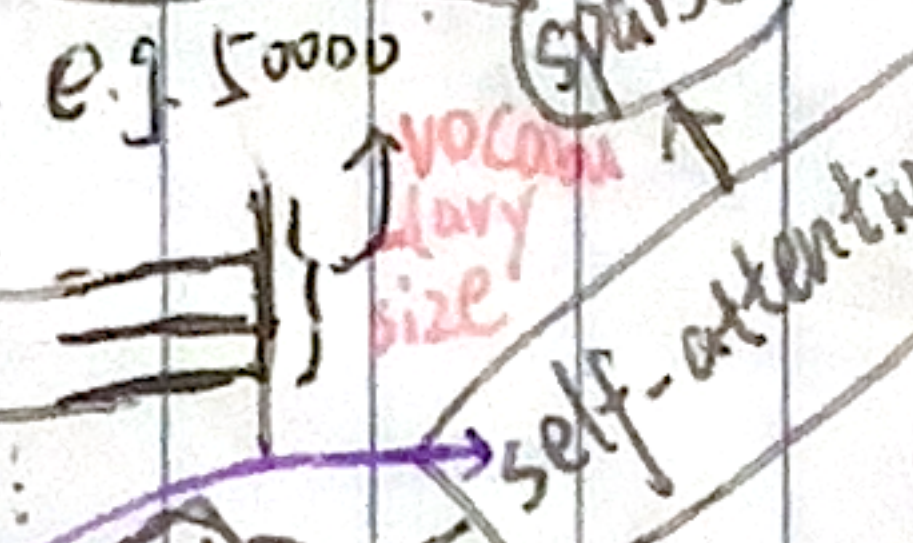
Token embedding

keep same structure.

Token matrix

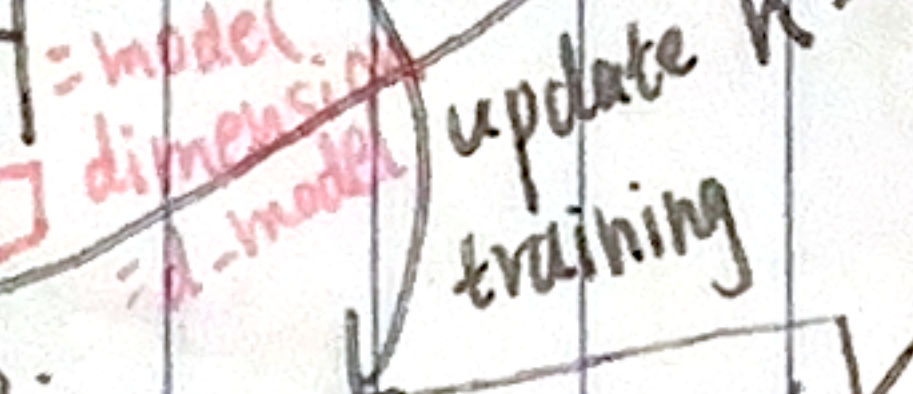
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



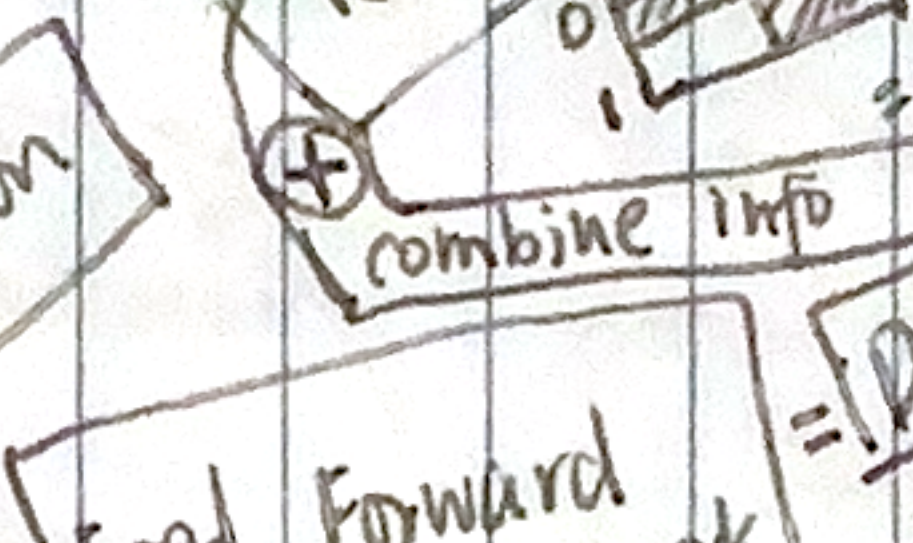
Token embedding

keep same structure.

Token matrix

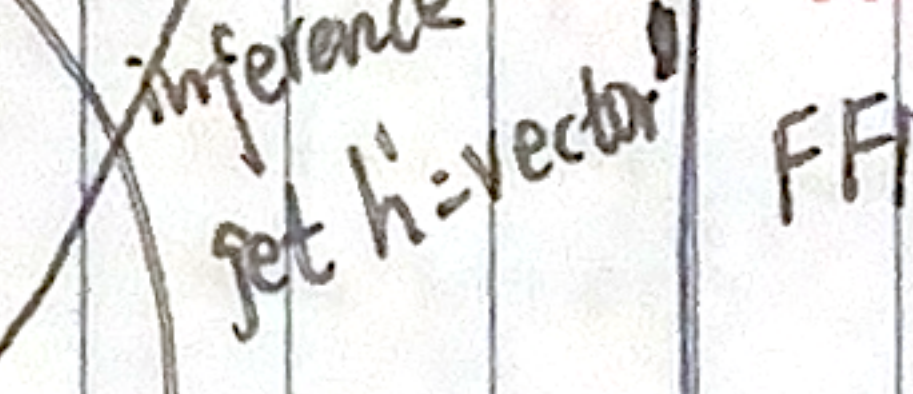
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



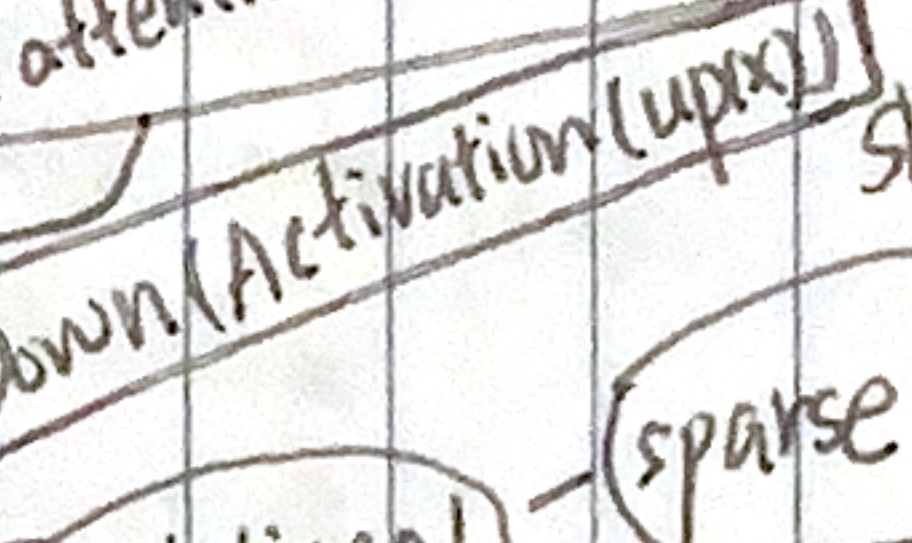
Token embedding

keep same structure.

Token matrix

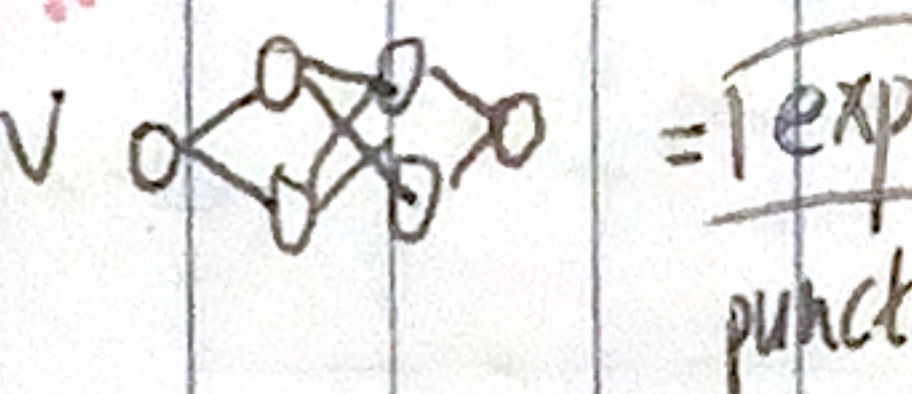
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



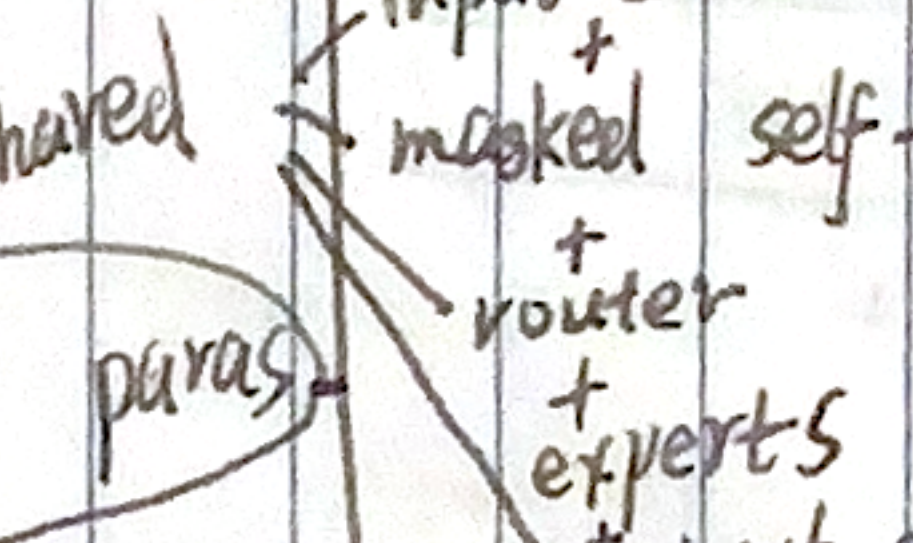
Token embedding

keep same structure.

Token matrix

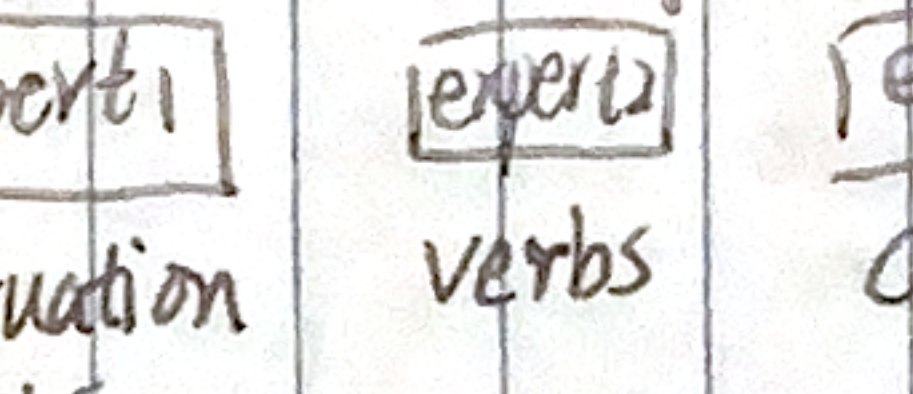
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



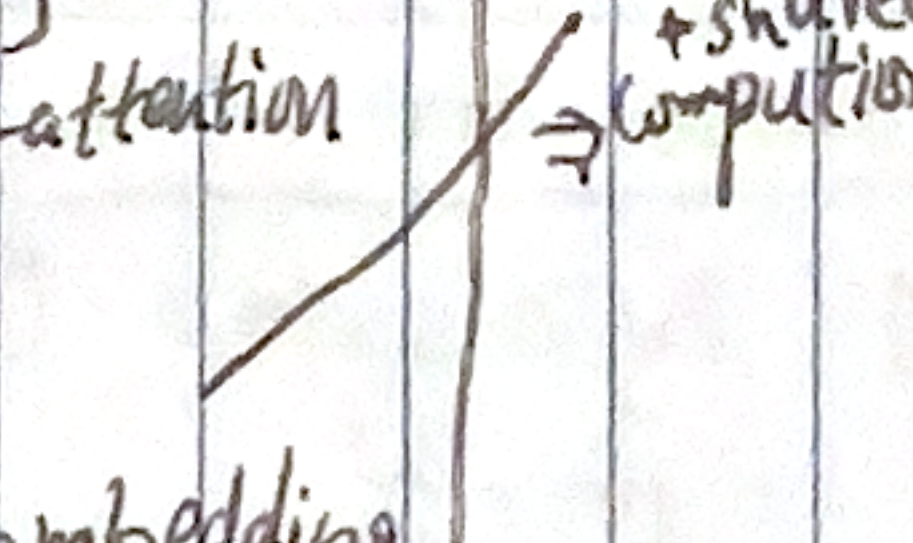
Token embedding

keep same structure.

Token matrix

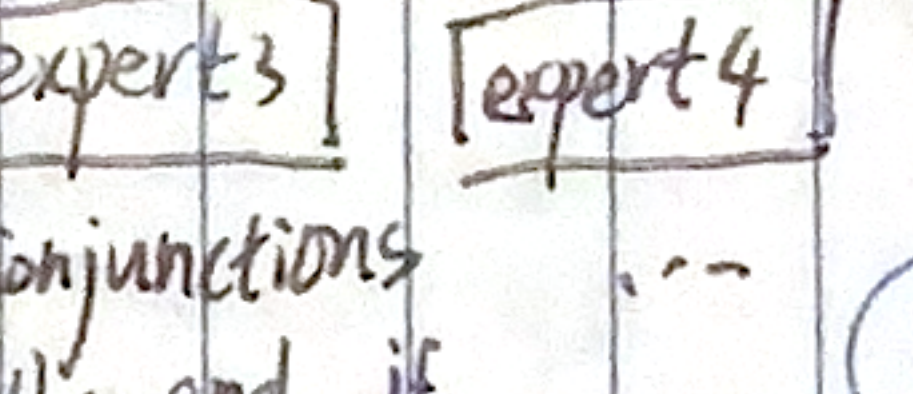
Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



Token embedding

keep same structure.

Token matrix

Find token based on tokenID

based on probabilities



token vocabulary e.g. 50000

based on tokenID find vector



Token embedding

keep same structure.

Token matrix

Find token based on tokenID

based on probabilities